

Round Robin Scheduling

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, t; // t is time quantum
```

```
    printf("Enter no. of processes: ");
    scanf("%d", &n);
```

```
    int AT[n], BT[n], RT[n];
```

```
    int CT[n], TAT[n], WTC[n];
```

```
    printf("Enter arrival time and burst time: \n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("p: %d AT: %d BT: %d", i+1);
```

```
        scanf("%d %d", &AT[i], &BT[i]);
```

```
        RT[i] = BT[i];
```

```
        CT[i] = 0;
```

```
    }
```

```
    printf("Enter time quantum: ");
```

```
    scanf("%d", &t);
```

```
    int time = 0;
```

```
    int completed = 0;
```

```
    int queue[100];
```

```
    int front = 0, rear = 0;
```

```
    int visited[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        visited[i] = 0; }
```

while (completed < n) {

for (int i = 0; i < n; i++) {

if (AT[i] <= time && visited[i] == 0) {

queue[rear++] = i;

visited[i] = 1;

}

}

if (front == rear) {

time++;

continue;

}

int p = queue[front++];

if (RT[p] > tq) {

time += tq;

RT[p] = tq;

for (int i = 0; i < n; i++) {

if (AT[i] <= time && visited[i] == 0) {

queue[rear++] = i;

visited[i] = 1;

}

}

queue[rear++] = p;

}

dec;

time += RT[p];

RT[p] = 0;

```
CT[p] = time;
complete++;
```

```
for (int i = 0; i < n; i++) {
```

```
    if (AT[i] <= time && visited[i] == 0) {
```

```
        queue[rear++] = i;
```

```
        visited[i] = 1;
```

```
    }
```

```
}
```

```
}
```

```
for (int i = 0; i < n; i++) {
```

```
    TAT[i] = CT[i] - AT[i];
```

```
    WT[i] = TAT[i] * B[i];
```

```
}
```

```
printf("In P.d | AT | BT | CT | TAT | WT |");
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("P.d | AT | BT | CT | TAT | WT |");
```

```
    i+1, AT[i], BT[i], CT[i], TAT[i], WT[i];
```

```
}
```

```
return 0;
```

```
}
```


Output:-

Enter number of processes : 4

Enter Arrival time and Burst time:

P1 AT BT : 0 5

P2 AT BT : 1 2

P3 AT BT : 2 4

P4 AT BT : 3 1

Enter Time Quantum : 2

PID	AT	BT	CT	TAT	WT
P1	0	5	12	12	7
P2	1	2	4	3	1
P3	2	4	11	9	5
P4	3	1	9	6	5

GAUNT Chart : P1 P2 P3 P4 P2 P3 P4 P2

Multilevel Queue Scheduling

```
#include <stdio.h>
```

```
int main()
```

```
int n;
```

```
printf("Enter number of processes:");  
scanf("%d", &n);
```

```
int Pid[n], AT[n], BT[n], Type[n];  
int CT[n], TAT[n], WT[n], start[n];
```

```
printf("Enter process details n");
```

```
for (int i=0; i<n; i++)
```

```
    Pid[i]=i+1;
```

```
    printf("Process %d\n", i+1);
```

```
    printf("Arrival Time");
```

```
    scanf("%d", &AT[i]);
```

```
    printf("Burst Time");
```

```
    scanf("%d", &BT[i]);
```

```
    printf("Type (0=System, 1=User):");
```

```
    scanf("%d", &Type[i]);
```

```
}
```

```
int system Q[n], user Q[n];
```

```
int syscount=0, user count=0;
```



```
for(int i=0; i<n; i++){
```

```
    if(Type[i]==0){
```

```
        systemA[systemAcount++] = i;
```

```
    }
```

```
    else{
```

```
        systemB[systemBcount++] = i;
```

```
    }
```

```
}
```

```
int time = 0;
```

```
for(int i=0; i<systemAcount; i++){
```

```
    int p = systemA[i];
```

```
    if(time < AT[p]){
```

```
        time = AT[p];
```

```
    }
```

```
    start[p] = time;
```

```
    time += BT[p];
```

```
    CT[p] = time;
```

```
    TAT[p] = CT[p] - AT[p];
```

```
    WTC[p] = TAT[p] - BT[p];
```

```
}
```

```
printf("The value of AT is AT is BT is CT is TAT is WTC is ");
```

```
for (int i=0; i<n; i++) {
```

```
    printf(" P %d 1st %d 2nd %d 3rd %d 4th %d 5th %d",
```

```
    i+1, AT[i], BT[i], CT[i], TAT[i], WT[i]);
```

```
    }
    return 0;
```

```
}
```

id	AT	CT	WT	id	AT	WT
0	2	2	0	2	0	11
1	2	4	1	3	1	11
2	2	4	0	4	2	11
3	1	1	1	5	2	11
4	1	1	0	6	2	11

OUTPUT:

Enter number of processes: 5

Arrival Time: 0

Burst Time: 5

Type (0=System, 1=User): 0

Process 2

Arrival Time: 1

Burst Time: 3

Type: 1

Process 3

Arrival Time: 2

Burst Time: 2

Type: 0

Process 4

Arrival Time: 4

Burst Time: 4

Type: 1

Process 5

Arrival Time: 3

Burst Time: 6

Type: 0

PID	AT	BT	Type	CT	TAT	WT
P1	0	5	0	5	5	0
P2	1	3	1	16	15	12
P3	2	2	0	7	5	3
P4	4	4	1	20	16	12
P5	3	6	0	13	10	4

GRANT FOR SYS A:

P1 P3 P5

GRANT FOR USER A:

P2 P4

Run
16/5/26